Bazar.com: Design and execution report

Distributed and Operating Systems - Lab 1 | Noor and Sara

Architecture and interfaces

Bazar.com is a two-tier bookstore with three independent Python microservices. Frontend is a REST gateway accepting user requests, while Catalog and Order form the backend. Each service is a separate process/container. The host command-line client sends HTTP requests to Frontend; all inter-service calls use HTTP and JSON. No graphical interface is required.

Search follows Client -> Frontend GET /search/<topic> -> Catalog GET /query/topic/<topic>. Catalog returns matching book IDs and titles. Information follows Frontend GET /info/<id> -> Catalog GET /query/item/<id>, returning ID, title, topic, price and available quantity. Topics match case-insensitively. Unknown topics return an empty array; unknown books return JSON HTTP 404.

Purchase follows Client POST /purchase/<id> -> Frontend -> Order. Order queries Catalog for book details and requests PATCH /update/<id> with stock_delta=-1. Catalog checks the latest stock inside a write transaction. Order saves a successful purchase in its own SQLite database and returns HTTP 201 with an order ID and title. Insufficient stock returns HTTP 409. The client and Order log the required bought book book_name message.

Persistent data

Catalog owns a books table: unique integer ID, title, topic, integer price_cents and quantity. The four titles are those specified by the assignment; books 1-2 are distributed systems and books 3-4 are undergraduate school. Initial quantities of ten and prices of 40, 50, 35 and 25 are chosen sample values. Startup inserts missing seed rows without resetting existing stock or prices.

Prices are stored in integer cents to avoid approximate monetary storage. API prices are JSON numbers in normal units. Decimal validates nonnegative incoming prices with at most two fractional digits. Database constraints reject negative prices and quantities. Order stores a generated ID, book ID, title, price snapshot and UTC creation time. Each backend opens and closes its own SQLite connections; databases are never shared between services.

Concurrency and update validation

PATCH /update/<id> accepts stock_delta, price, or both. Positive deltas restock a title; negative deltas remove copies. Empty/malformed requests, unknown fields, boolean or fractional deltas, unsupported integer ranges and invalid prices produce JSON HTTP 400. A rejected combined update leaves both stock and price unchanged.

Catalog begins BEGIN IMMEDIATE before reading quantity. SQLite reserves the write transaction, forcing competing updates to wait rather than all reading the same old stock. The service checks the proposed quantity, writes both fields and commits together. A decrease below zero returns 409. Thus an earlier availability query cannot authorize a stale decrement. Waitress supplies concurrent worker threads in Docker, while stock consistency is enforced by the database transaction.

Failure behavior and tradeoffs

Frontend preserves backend JSON/status and bounds socket waits to five seconds. Order uses bounded Catalog requests. Connection failures return JSON 503; invalid upstream JSON produces a gateway 502. Requests are not automatically retried. A timeout can occur after a server committed, so failure to receive a response does not prove that a purchase did not happen.

Catalog stock safety is not a distributed all-or-nothing purchase. If order insertion fails after a decrement, Order attempts a compensating stock increment. Compensation can fail or time out; a crash can occur before it is attempted. Commit or network ambiguity can require manual reconciliation. A lookup-time price snapshot can also become stale if an operator changes the price during a purchase. These limitations are explicit rather than presented as exactly-once processing.

SQLite and lightweight Flask services suit the small lab, avoiding heavyweight database/framework administration. SQLite serializes writers and may encounter lock contention under heavy loads. Named volumes preserve data, but they are not backup or disaster-recovery mechanisms. Frontend routes by operation; it does not balance traffic across replicas. There is no authentication, payment processing, cancellation or shipping workflow, and inventory administration is intended for trusted operators.

Improvements and extensions

A stronger design would use client-generated purchase keys, durable Catalog reservations and a unique matching Order receipt. Repeated keys could return the original outcome, while a reconciliation worker completes pending orders or releases abandoned reservations. A reservation could carry the authoritative purchase price. Authentication for inventory updates, structured request IDs, latency/error metrics, backups and an idempotent delivery feed are further extensions. Automatic restock is not fabricated; operators apply real delivery changes through the update API.

Running and deploying the program

Clone branch noor, enter the repository and run docker compose up --build -d --wait with Docker Desktop in Linux-container mode. docker compose ps shows the three services. Each container runs Waitress on 0.0.0.0; Catalog is catalog:5001, Order is order:5002, and Frontend is frontend:5000 inside the private network. Only Frontend is published to host loopback port 5000. FRONTEND_PORT can select another host port. Health checks sequence startup.

Catalog mounts its named volume at /app/data; Order mounts a separate volume at /data. docker compose down removes containers but retains these volumes. docker compose up -d --wait recreates containers with saved data. Adding -v intentionally deletes the databases. The build context allows only service sources and dependency/build files, excluding local data, Git history, environments and unrelated directories.

For the host client, create .venv with python -m venv .venv and install requirements using .venv/Scripts/python.exe -m pip install -r requirements.txt on Windows. Run client.py search "distributed systems", client.py info 2 or client.py purchase 2 through that interpreter. --base-url selects another Frontend address. Search/info print readable results; purchase prints bought book followed by the actual title. The client remains outside Docker.

For local development, run catalog.py, order.py and frontend.py in separate terminals on ports 5001, 5002 and 5000. These entry points use Flask's development server. Configure reachable backend URLs and an appropriate server bind when using separate machines. Catalog and Order storage paths can be set through environment variables; each service retains ownership of its SQLite file. The README includes exact commands, inventory updates and log inspection.

Testing and measured results

The combined regression suite passed 23 tests: three Catalog, fourteen Order and six Client tests. They cover stock safety, purchases, persistence, concurrency, dependency failures, compensation and client behavior. verify_noor.py verifies Catalog/gateway behavior using a clearly identified Order test double for routing; it is not evidence of real order processing.

verify_docker.py builds the current integrated sources in a temporary context and runs three real Linux containers with disposable volumes. The host client searched both categories and bought book 2, observing stock drop from ten to nine. Thirty real concurrent purchases for ten copies yielded ten HTTP 201 responses, twenty HTTP 409 responses and zero remaining stock. Container removal/recreation preserved stock and eleven orders. Restock, price change, another purchase and unavailable-Order JSON 503 behavior were also verified.

Actual test, client, build, status and component output is retained in docs. All services log request outcomes, and Order logs purchased titles. Test containers/volumes are removed afterward without altering normal data. Git history preserves both contributors' work and separate integration, deployment, verification and documentation commits. The source, report and execution evidence are delivered together on noor.